

Фамилия: Филибер

Имя: Никола

Группа: НФИмд-02-25

Адрес электронной почты: 1132259430@rudn.ru

Математические основы защиты
информации и информационной
безопасности

Лабораторная работа N°4

Вычисление наибольшего общего делителя

1. Алгоритм Евклида.

Для вычисления наибольшего общего делителя двух целых чисел

10, 45

$$45 = 10 * q + r$$

$$45 = 10 * 4 + 5$$

$$10 = 5 * 2 + 0$$

5 -> наибольшего общего делителя

```
# Euclid's Algorithm (remainder / division method)
def gcd_euclid(a: int, b: int) -> int:
    """Classic Euclid GCD using remainders."""
    a, b = abs(a), abs(b)
    if a == 0:
        return b
    if b == 0:
        return a

    while b != 0:
        a, b = b, a % b
    return a

gcd = gcd_euclid(10, 45)
print("GCD", gcd)
```

2. Бинарный алгоритм Евклида.

Вход. Целые числа a, b ; $0 < b \leq a$.

Выход. $d = \text{НОД}(a, b)$.

1. Положить $g \leftarrow 1$.

2. Пока оба числа a и b четные, выполнять $a \leftarrow a/2$, $b \leftarrow b/2$, $g \leftarrow 2g$ до получения хотя бы одного нечетного значения a или b .

3. Положить $u \leftarrow a$, $v \leftarrow b$.

4. Пока $\neq 0$ выполнять следующие действия:

4.1. Пока u четное, полагать $u \leftarrow u/2$

4.2. Пока v четное, полагать $v \leftarrow v/2$

4.3. При $u \geq v$ положить $u \leftarrow u - v$. В противном случае положить $u \leftarrow v - u$

5. положить $d \leftarrow gv$

6. результат: d

```
#Binary GCD (Stein's algorithm)
def gcd_binary(a: int, b: int) -> int:
    """Binary GCD (Stein's algorithm)."""
    a, b = abs(a), abs(b)
    if a == 0:
        return b
    if b == 0:
        return a

    # count common factors of 2
    shift = 0
    while ((a | b) & 1) == 0: # both even
        a >>= 1
        b >>= 1
        shift += 1

    # make a odd
    while (a & 1) == 0:
        a >>= 1

    while b != 0:
        # make b odd
        while (b & 1) == 0:
            b >>= 1

        # now a and b are odd
        if a > b:
            a, b = b, a
        b = b - a # b becomes even (difference of odds)

    return a << shift

gcd = gcd_binary(45, 10)
print(gcd) # 5
```



```
#Extended Euclid (returns gcd, x, y)
def extended_gcd_euclid(a: int, b: int):
    """
    Returns (g, x, y) such that:
        g = gcd(a, b)
        a*x + b*y = g
    """
    if b == 0:
        g = abs(a)
        # make x carry the sign of a so identity holds
        x = 1 if a > 0 else -1 if a < 0 else 0
        return g, x, 0

    old_r, r = a, b
    old_x, x = 1, 0
    old_y, y = 0, 1

    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_x, x = x, old_x - q * x
        old_y, y = y, old_y - q * y

    g = abs(old_r)
    # Normalize sign so g is positive
    if old_r < 0:
        old_x, old_y = -old_x, -old_y

    return g, old_x, old_y
```